

Design and Implementation of an 8-bit Pipelined Processor Using Verilog HDL and AMD Xilinx Vivado

Nigam Mehta

mehtanigam3024@gmail.com

Pursuing B.Tech in Electronics Engineering (VLSI Design and Technology)

Abstract—This paper presents the design, simulation, synthesis, and implementation analysis of an 8-bit pipelined processor developed using Verilog HDL and AMD Xilinx Vivado. The processor uses a compact instruction set consisting of ADD, ADDI, SUB, SLL, and HALT instructions. The datapath is organized into instruction-fetch (IF), instruction-decode (ID), and execute (EX) stages separated by IF/ID and ID/EX pipeline registers. The design contains an 8-bit program counter, a 16-location instruction memory, an 8×8-bit register file with register R0 fixed to zero, a control unit, an 8-bit ALU, hazard detection logic, pipeline stalling and flushing, and a latched HALT output. A Verilog testbench was used with a 100-MHz clock to verify instruction execution and pipeline activity. The supplied Vivado results show a final simulation LED value of 00001010 and a HALT flag of 1. Post-implementation timing reports show a worst negative setup slack of 5.123 ns with zero total negative slack, while synthesis reports show 95 slice LUTs and 104 slice registers used. RTL and synthesized schematics are included, with the large schematic pages split into readable image halves for inclusion in the IEEE-format report.

Index Terms—Pipelined processor, Verilog HDL, FPGA, Vivado, ALU, hazard detection, pipeline stall, pipeline flush, RTL synthesis, XSim.

I. INTRODUCTION

Processor pipelining divides instruction execution into multiple stages so that different instructions can occupy different stages of the datapath at the same time. This improves instruction throughput compared with a single-cycle design, although it introduces control and data dependencies that must be handled correctly.

This project implements a compact 8-bit processor in Verilog HDL. The design is intended to demonstrate the fundamental concepts of a pipelined datapath, including instruction fetching, decoding, arithmetic and logical execution, register updates, pipeline registers, RAW hazard detection, stalling, flushing, and HALT handling. The complete project was developed and verified using AMD Xilinx Vivado and Vivado XSim.

II. OBJECTIVES

The objectives of the project are:

- 1) Design an 8-bit pipelined processor using synthesizable Verilog HDL.
- 2) Implement a program counter and instruction memory.
- 3) Implement an 8×8-bit register file.
- 4) Design a control unit and 8-bit ALU.

TABLE I
MAIN PROCESSOR COMPONENTS

Component	Function
PC	8-bit instruction address generation
Instruction memory	Stores 16 test instructions
IF/ID register	Pipeline boundary between IF and ID
Control unit	Opcode decoding and control generation
Register file	Eight 8-bit registers, R0 fixed to zero
ID/EX register	Pipeline boundary between ID and EX
ALU	ADD, SUB, AND, and SLL operations
Hazard logic	Detects selected RAW dependency and requests stall
Flush logic	Clears pipeline state for HALT handling
LED register	Holds the latest valid ALU result

- 5) Insert pipeline registers between processor stages.
- 6) Detect a selected class of read-after-write (RAW) data hazards.
- 7) Implement pipeline stalling and HALT-related flushing.
- 8) Verify the processor through behavioral simulation.
- 9) Perform synthesis and implementation in Vivado.
- 10) Analyze RTL, synthesized architecture, resource utilization, and timing.

III. PROCESSOR ARCHITECTURE

The implemented datapath contains three principal stages:

- 1) **Instruction Fetch (IF):** The program counter addresses instruction memory.
- 2) **Instruction Decode (ID):** The instruction is decoded, register operands are read, and control signals are generated.
- 3) **Execute (EX):** The ALU performs the selected arithmetic or logical operation. The result is written back to the register file through the synchronous write port.

The two explicit pipeline boundaries are the IF/ID and ID/EX registers. The top-level data flow can be summarized as

$$PC \rightarrow IMEM \rightarrow IF/ID \rightarrow Decode/RF \rightarrow ID/EX \rightarrow ALU. \quad (1)$$

The ALU result is also connected to an LED register for external observation. The HALT condition is latched into `halt_flag`.

TABLE II
INSTRUCTION SET

Instruction	Encoding	Operation
ADD	00	$R_d = R_d + R_s$
ADDI	01	$R_d = R_d + \text{Imm}$
SUB	10	$R_d = R_d - R_s$
SLL	11	$R_d = R_d \ll \text{Imm}$
HALT	11111111	Stop indication

IV. INSTRUCTION SET AND ENCODING

Instructions are 8 bits wide and use the format

$$[\text{opcode}_{7:6}][R_{d5:3}][R_s/\text{imm}_{2:0}]. \quad (2)$$

The implemented instruction set is shown in Table II.

For ADD and SUB, the decoded destination register is also used as the first source operand. The register file is therefore connected so that `rs1` receives `id_rd` and `rs2` receives `id_rs`.

V. ALU DESIGN

The 8-bit ALU supports four control codes:

$$\text{ALU}(A, B) = \begin{cases} A + B, & 00 \\ A - B, & 01 \\ A \& B, & 10 \\ A \ll B[2:0], & 11. \end{cases} \quad (3)$$

A zero flag is generated as

$$Z = (\text{result} = 0). \quad (4)$$

The ALU is combinational and therefore does not itself introduce a clocked stage. Its result is consumed by the sequential register-write path and by the LED output register.

VI. REGISTER FILE

The register file contains eight registers of eight bits each:

$$RF = \{R_0, R_1, \dots, R_7\}. \quad (5)$$

Register R_0 is hardwired to zero. Writes to register zero are ignored, while asynchronous reads of register zero return zero. Other registers are written synchronously on the positive clock edge when `reg_write` is asserted.

VII. PIPELINE REGISTERS

The IF/ID register stores the fetched instruction and current program-counter value. It supports both stall and flush behavior.

The ID/EX register stores operand values, destination register, immediate value, and control signals:

$$\{D_1, D_2, R_d, \text{Imm}, \text{RegWrite}, \text{ALUSrc}, \text{ALUCtrl}, \text{Halt}\}. \quad (6)$$

On reset or flush, the register outputs are cleared, effectively inserting a bubble into the corresponding stage.

VIII. HAZARD DETECTION AND STALLING

A RAW dependency occurs when a later instruction needs a value that is still associated with the instruction currently in the EX stage. The top-level logic implements:

$$\text{stall}_{\text{raw}} = \text{ex_reg_write} \wedge (\text{ex_rd} \neq 0) \wedge (\text{ex_rd} = \text{id_rs}). \quad (7)$$

HALT-related flushing takes priority over the stall:

$$\text{stall}_{\text{final}} = \text{stall}_{\text{raw}} \wedge \neg \text{flush}_{\text{halt}}, \quad (8)$$

where

$$\text{flush}_{\text{halt}} = \text{ex_halt}. \quad (9)$$

When a stall is asserted, the PC and IF/ID register hold their current values. When a flush is asserted, the corresponding pipeline registers are cleared. This demonstrates the basic mechanism required to prevent selected data dependencies from progressing incorrectly through the pipeline.

IX. HALT HANDLING

The instruction memory reserves 11111111 as HALT. When HALT reaches the EX stage, `ex_halt` becomes active. This signal is used to flush pipeline state and is also latched into the external `halt_flag` output:

$$\text{halt_flag}_{k+1} = \text{halt_flag}_k \vee \text{ex_halt}. \quad (10)$$

The PC receives the EX-stage halt indication and is designed to freeze while HALT is active.

X. INSTRUCTION MEMORY AND TEST PROGRAM

The instruction memory contains 16 entries addressed using the lower four bits of the 8-bit program counter. The test program exercises immediate arithmetic, register arithmetic, subtraction, shifting, and HALT.

Representative instructions include:

```
memory[0] = 8'b01_001_011; // ADDI R1, 3
memory[1] = 8'b01_010_101; // ADDI R2, 5
memory[2] = 8'b00_001_010; // ADD R1, R2
memory[3] = 8'b10_010_001; // SUB R2, R1
memory[6] = 8'b11_011_001; // SLL R3, 1
memory[15] = 8'b11111111; // HALT
```

The program therefore exercises the ALU with positive and two's-complement 8-bit values as well as shift operations.

XI. SIMULATION METHODOLOGY

Behavioral simulation was performed with Vivado XSim using the supplied `tb_top.v` testbench. The clock has a 10-ns period:

$$T_{\text{clk}} = 10 \text{ ns}, \quad f_{\text{clk}} = 100 \text{ MHz}. \quad (11)$$

The testbench applies reset for 20 ns and then allows the processor to execute for 500 ns. A monitor prints the PC, ID instruction, EX ALU result, LED value, and HALT flag at every relevant simulation event.

The supplied simulation log reports:

- simulation completion at 520 ns;
- final LED output = 00001010 (hexadecimal 0A); and

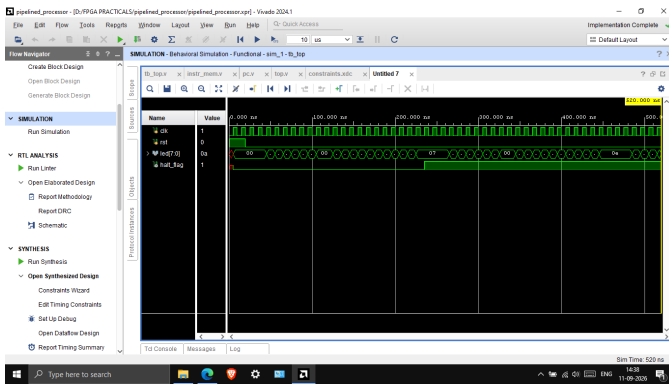


Fig. 1. Supplied Vivado XSim simulation waveform.

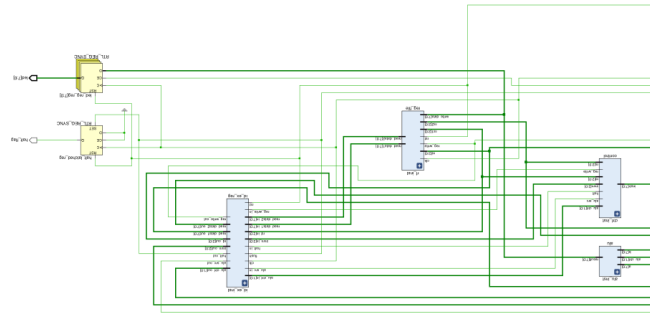


Fig. 2. RTL schematic, left half.

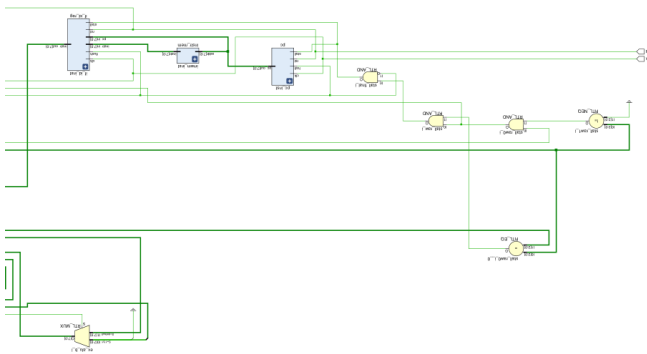


Fig. 3. RTL schematic, right half.

- final HALT flag = 1.

The waveform demonstrates instruction progression, repeated PC values during stall events, ALU result changes, LED updates, and the asserted HALT state.

XII. RTL SCHEMATIC

The supplied Vivado RTL schematic is a generated view of the synthesized RTL connectivity. Because the original schematic is large for a single IEEE column, it is converted into two readable halves for this report.

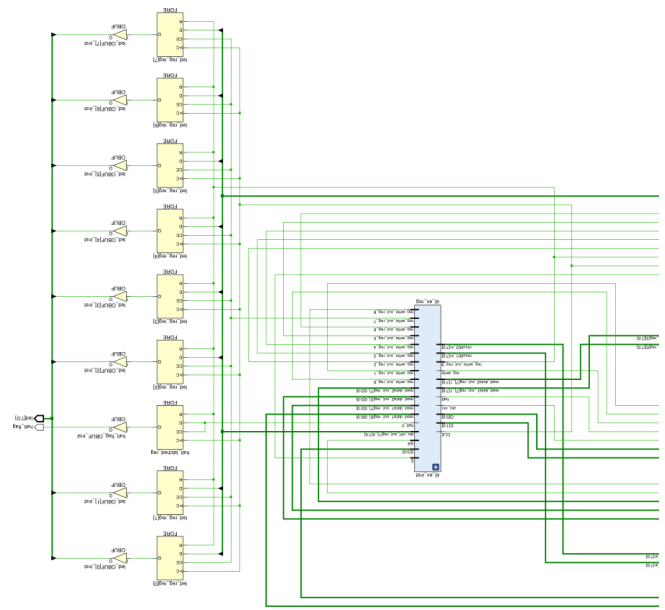


Fig. 4. Synthesized schematic, left half.

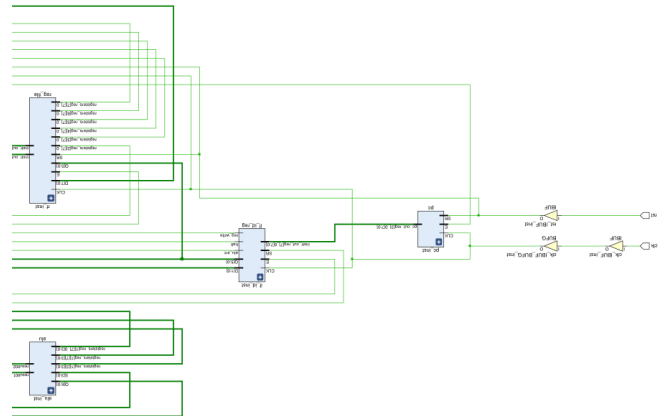


Fig. 5. Synthesized schematic, right half.

The RTL view confirms the presence and interconnection of the PC, instruction memory, pipeline registers, register file, control logic, ALU, and output/hazard logic.

XIII. SYNTHESIZED SCHEMATIC

After synthesis, Vivado generates a technology-oriented schematic composed of FPGA primitives and optimized logic. The original supplied synthesized schematic is similarly divided into two halves to preserve readability.

TABLE III
SYNTHESIS RESOURCE UTILIZATION

Resource	Used	Utilization
Slice LUTs	95	1.19%
Slice Registers	104	0.65%
F7 Muxes	16	0.40%

The synthesized view confirms that the Verilog design can be mapped into FPGA resources. The implementation reports also provide resource and timing information.

XIV. SYNTHESIS AND IMPLEMENTATION RESULTS

The supplied Vivado reports show the following resource utilization:

The design is therefore small relative to the available FPGA fabric reported by Vivado. The low utilization is consistent with the compact 8-bit datapath and small instruction memory.

A. Timing Analysis

The project uses a 10-ns clock constraint. The final routed timing report shows:

$$WNS = 5.123 \text{ ns}, \quad (12)$$

with total negative slack

$$TNS = 0 \text{ ns}. \quad (13)$$

The corresponding worst-case setup data-path delay is approximately 4.748 ns for the reported critical path, leaving positive setup margin against the 10-ns requirement.

The positive WNS and zero TNS indicate that the implemented design meets the specified 100-MHz clock constraint in the supplied Vivado implementation run.

XV. FPGA CONSTRAINTS

The supplied XDC file specifies a 10-ns clock and I/O constraints for the target FPGA package. The design uses:

- clock input on A1;
- reset input on A2;
- eight LED output pins on B1, B2, C1, C2, D1, D2, E1, and E2; and
- HALT output on F1.

All listed I/O ports use the LVCMOS33 standard. The XDC comments identify the target as an XA7S15CSGA196-1L device/package configuration.

XVI. TESTING AND RESULTS

The testbench exercises the processor after reset and observes internal and external signals. Representative checks are summarized in Table IV.

The simulation log shows multiple PC hold intervals, for example at 35–55 ns and 145–155 ns, which are consistent with stall activity. The final reported LED output is hexadecimal 0A and the HALT flag is asserted.

TABLE IV
SIMULATION VERIFICATION

Test	Observed Result
Reset	PC and pipeline state initialized
ADDI	Immediate arithmetic produces ALU results
ADD	Register operands processed by ALU
SUB	8-bit subtraction including wrap-around values
SLL	Left-shift operation observed
Pipeline stall	PC/IF-ID hold during selected RAW condition
Pipeline flush	Pipeline registers cleared on flush condition
HALT	HALT flag latched to 1
LED output	Final value 00001010
Synthesis	Design successfully synthesized
Implementation	Positive WNS and zero TNS

XVII. DISCUSSION

The project demonstrates the fundamental relationship between pipeline registers, control signals, hazards, and throughput. The explicit IF/ID and ID/EX registers allow instructions to occupy different portions of the datapath concurrently. The small instruction set keeps the control unit compact, while the register file and ALU provide a reusable execution datapath.

A notable implementation feature is that the design exposes both the LED result and HALT status as top-level outputs. This makes the processor easier to demonstrate on FPGA hardware or through simulation.

The supplied synthesis and implementation results also show that the architecture is lightweight. Only 95 LUTs and 104 slice registers are reported as used, and the design satisfies the supplied 100-MHz timing requirement with substantial positive setup slack.

XVIII. LIMITATIONS

The current implementation is intentionally compact and has several limitations:

- 1) The processor has only three principal pipeline stages rather than a full five-stage RISC pipeline.
- 2) The instruction memory contains only 16 locations.
- 3) The instruction set does not include load/store, branch, jump, or memory-data instructions.
- 4) The hazard detector implements a limited RAW dependency check rather than a complete forwarding and hazard-resolution network.
- 5) There is no branch prediction or control-hazard subsystem.
- 6) The test program is initialized directly in Verilog instruction memory.
- 7) The processor is an educational RTL design rather than a complete general-purpose CPU.

XIX. FUTURE SCOPE

The processor can be extended by adding:

- 1) a five-stage IF/ID/EX/MEM/WB pipeline;

- 2) forwarding paths from later stages to the ALU inputs;
- 3) comprehensive hazard detection for all source operands;
- 4) branch and jump instructions with control-hazard handling;
- 5) data memory and load/store instructions;
- 6) a larger instruction memory and program-loading mechanism;
- 7) interrupt and exception support;
- 8) additional ALU operations and status flags;
- 9) performance counters for CPI and stall analysis; and
- 10) FPGA board-level verification using switches, LEDs, and a clock divider.

XX. CONCLUSION

An 8-bit pipelined processor was designed and implemented in Verilog HDL using AMD Xilinx Vivado. The architecture contains instruction fetch, instruction decode, and execute stages with IF/ID and ID/EX pipeline registers, an 8-bit ALU, an 8×8-bit register file, instruction memory, control logic, selected RAW hazard detection, stalling, flushing, and HALT handling.

The supplied Vivado XSim simulation successfully exercised the processor and reported a final LED output of 00001010

with the HALT flag asserted. Synthesis and implementation were also completed successfully. The reported 95 LUTs and 104 slice registers demonstrate low resource consumption, while the 5.123-ns WNS and zero TNS demonstrate that the design meets the 10-ns clock constraint in the supplied implementation run.

Overall, the project provides a compact but complete demonstration of RTL processor design, pipeline operation, hazard management, simulation, synthesis, and FPGA timing analysis.

ACKNOWLEDGMENT

The author acknowledges the academic laboratory environment and AMD Xilinx Vivado/XSim tools used for the design, simulation, synthesis, and implementation of the processor.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.
- [2] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, Morgan Kaufmann.
- [3] AMD, "Vivado Design Suite User Guide," AMD Xilinx documentation.
- [4] IEEE Standard for Verilog Hardware Description Language, IEEE Std. 1364.